DL Homework 2

José Machado
IST 1106643
jose.g.machado@tecnico.ulisboa.pt

Gil Jorge
IST 110062
gil.jorge@tecnico.ulisboa.pt

Elias Heimdal
IST 116702
elias.heimdal@tecnico.ulisboa.pt

Abstract

For Question 1, José Machado was responsible for the implementation of the code, generation of all plots, and the writing of the corresponding report section, summarizing all the contents related to Question 1.

For Question 2, Elias Heimdal was responsible for the design of both models architectures, and train, evaluations, data loading, and hyperparameter search codes. José Machado was responsible for the creation of variations of HW2_Q2_1_ notebooks, which save different grid searches in different output files, and the writing of Question 2 subsections 2.1 - 2.1.3 .

Gil revised the code for 2.1, expanded it doing question 2.2, namely implementation of the code, generation of all plots, writing of the corresponding report section, and solving 2.3.

1 Question 1

Using the code in `hw2_q1.py`, four convolutional neural network (CNN) were trained on the BloodMNIST dataset, which contains 17,092 images and 8 classes, for 200 epochs. Validation and training losses and accuracies are saved in `model_train_metrics.json`, and training time in `time_measures.json`. Images were split with a ratio of 7:1:2 into training, validation, and test set. Each of the CNN's is built as specified in the Homework assignment, where each model results of the possible combinations of having or not max pooling after the convolution layers, and softmax activation in last linear layer.

Model	Max Pooling	Softmax Output
Model 1	No	No
Model 2	No	Yes
Model 3	Yes	No
Model 4	Yes	Yes

Table 1: Summary of CNN architectures.

1.1 Logits × Softmax

The first two models returned the following test accuracy of the model with the best performance on the validation set, and the respective epoch:

Model	Best Epoch	Val. Acc.	Test Acc.
1	23	0.939	0.935
2	9	0.430	0.435

Table 2: Best epoch, validation, and test accuracies for models without max pooling layers

Model 1 got a considerably better results in both accuracies, with differences of 0.509 and 0.500 on validation and test accuracies, allowing to conclude that the model returning logits performed significantly better than the model with softmax activation.

The script `hw2_q1_1_plots.py` created the following plots showing the training loss and validation accuracy for both models:

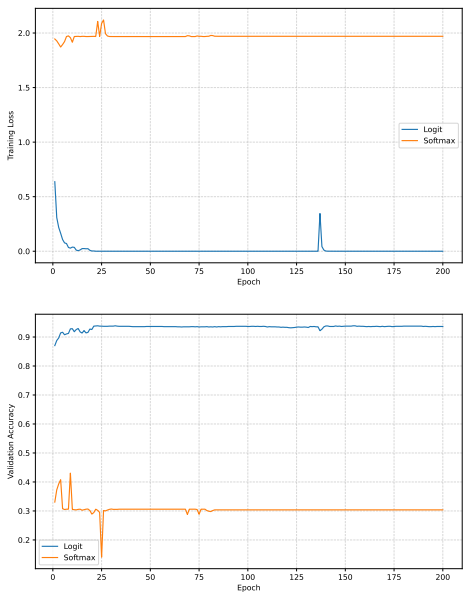


Figure 1: Plot 1: loss over epochs for both models. Plot 2: validation accuracy over epochs

Its clear that Model 2 is having regression problems, and the cause can be attributed to applying `softmax + nn.CrossEntropyLoss()`, which inside already applies the softmax operation, leading to the model applying two softmax in sequence, which can make the gradient become small and vanish, significantly hindering the model ability to regress.

1.2 Max Pooling

1.2.1 effectiveness

Models 3 and 4 returned the following test accuracy of the model with the best performance on the validation set, and the respective epoch:

Model	Best Epoch	Val. Acc.	Test Acc.
3	60	0.954	0.953
4	133	0.946	0.933

Table 3: Best epoch, validation, and test accuracies for models with max pooling layers.

This table shows that including max pooling layers improved performance on both validation and test sets, for both models 1 and 2. The model with softmax got a remarkable better performance of 0.516 and 0.498 in validation and test accuracies, indicating that max pooling significantly improved the model's ability to learn robust features and generalize, leading to effective convergence during training.

1.2.2 efficiency

By reading times saved in `time_measures.json`, we get the following table displaying the time each model took to run for 200 epochs, and by calculating $(t_{total}/200) * BestEpoch$, we also get the expected time it took to reach the epoch with best validation accuracy:

Model	Time (s)	Best Epoch Time (s)
Model 1	1365.05	156.98
Model 2	1320.04	39.60
Model 3	755.72	226.716
Model 4	767.54	510.4141

Table 4: Summary of total CNN training time and time to reach best performing epoch.

Models with max pooling (3 and 4) have substantially lower total training times, being approximately 1.81x and 1.72x faster than their counterparts without max polling (1 and 2), respectively. This shows that max pooling reduces computational cost by decreasing the spacial resolution of the feature maps.

However, model 1 reached its best validation accuracy earlier than model 3, indicating that max pooling may require more epochs to reach optimal performance. Model 2 has a much earlier best epoch compared to model 4, but model 2 has very poor performance.

1.3 Overall performance

Models 1, 3, and 4 achieved similar test accuracies, differing by t most 2%. However, model 1 reached its best validation epoch $1.44\times$ and $3.25\times$ faster than models 3 and 4, respectively, indicating to be the best trade-off between accuracy and training time. Model 3, while slightly slower to converge compared to model 1, achieved the highest test accuracy. Therefore, model 1 is most suitable when efficiency is considered, while model 3 is most suitable when only effectiveness is considered.

2 Question 2

2.1 Vanilla CNN and BiLSTM

This work models the binding affinity between the protein RBFOX1 and RNA strands. While RBFOX1 recognizes a short specific RNA sequence of 5-6 nucleotides, the whole sequence influences the overall binding affinity due to structural and chemical interactions between full RNA strand and protein. Thus, the problem requires modeling both the local binding region and the global RNA sequence context. (1)

To capture local sequence patterns, a convolution neural network (CNN) is used. CNN's have the ability to find regional patterns on the input, making them suitable to find the local RNA sequences that directly bind with the protein. The CNN architecture is based on three convolution layers, each with kernels size 5, 7, and 9, allowing detection of core binding sequences and regional patterns.

To model long-range dependencies across the full RNA strand, a bidirectional Long Short-Term Memory (BiLSTM) network is used. LSTM's have the ability to capture global patterns and long-range interactions, and the bidirectional design reflects RNA directionality ($5' \rightarrow 3'$ and $3' \rightarrow 5'$), enabling the model to process nucleotide information for both orientations. The BiLSTM then accounts for the influence of the full strand on binding affinity. (2)

2.1.1 First model

The model consists of a CNN with convolution layers, each followed by a batch normalization layer (BNL). The output of the last BNL is passed through both max and average pooling layers, followed by two fully connected layers. ReLU activations are applied after each BNL and linear layers.

The tuned hyperparameters were the learning rate, dropout rate, and the width of the first fully connected layer. To manage the model's considerable training time, an iterative grid search was used, based on successive small grids either exploring promising parameters or wider ranges. Each grid was run with `HW2_Q2_1_t*.ipynb`, saving results on `Outputs_t*/CNN_search_results_t*.json`. Figure 2, generated with `plot_full_grid.py`, shows the validation correlations for the different hyperparameter combinations.

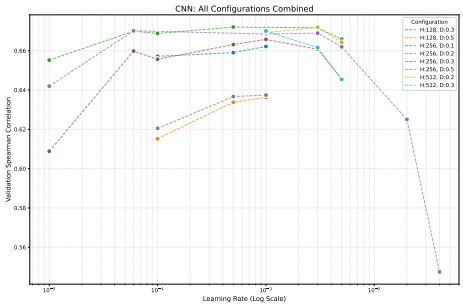
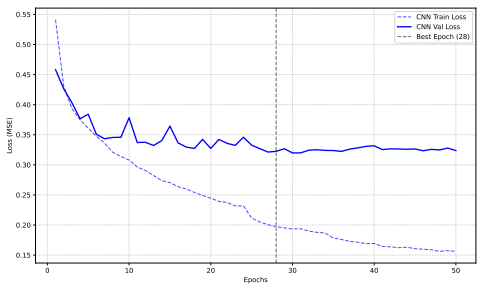


Figure 2: Iterative grid search for CNN architecture, considering models with validation correlation higher than 0.3

Following information comes from `get_bestModels.ipynb`. The hyperparameters of the best model are: $\boldsymbol{lr} = 5 \times 10^{-4}$, $\boldsymbol{dr} = 0.1$, $\boldsymbol{H} = 256$. The model registered validation and test correlation of $\mathbf{0.672}$ and and $\mathbf{0.665}$ in epoch $\mathbf{28}$. The model's losses and correlations are shows in the following figures, with a vertical line highlighting best epoch:



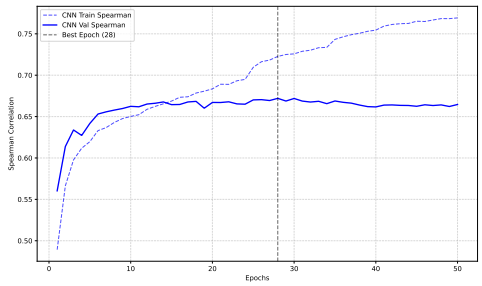


Figure 3: Plot 1: training and validation sets loss over epochs. Plot 2: training and validation Spearman correlations over epochs. Plots from CNN model.

2.1.2 Second model

The model consists of a two-layer BiLSTM that processes one-hot encoded RNA sequences, followed by batch normalization and dropout for regularization. The final sequence representation is obtained by concatenating the last hidden states from the forward and backward LSTM directions. This representation is then passed through two fully connected layers with a ReLU activation.

The tuned hyperparameters were the learning rate, dropout rate, and the width of the first fully connected layer. Just like for the CNN, an iterative grid search was run, which is based on the same files `HW2_Q2_1_t*.ipynb`, with results saved in `Outputs_t*/LSTM_search_results_t*.json`.

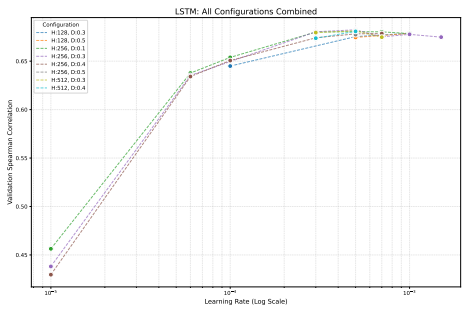


Figure 4: Iterative grid search for LSTM architecture, considering models with validation correlation higher than 0.3

Following information comes from `get_bestModels.ipynb`. The hyperparameters of the best model are: $\boldsymbol{lr} = 5 \times 10^{-4}$, $\boldsymbol{dr} = 0.3$, $\boldsymbol{H} = 256$. The model registered validation and test correlations of $\mathbf{0.682}$ and $\mathbf{0.681}$ in epoch $\mathbf{48}$. The model's losses and correlations are shows in the following figures, with a vertical line highlighting best epoch:

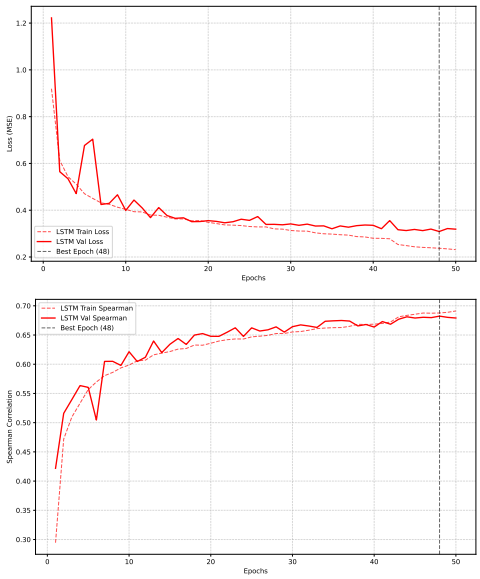


Figure 5: Plot 1: training and validation sets loss over epochs. Plot 2: training and validation spearman correlations over epochs.

2.1.3 Comparative results.

Comparing the two models of the different architectures that have highest validation correlation, the LSTM model has a higher validation correlation by a difference of $\mathbf{0.01}$ ($\mathbf{1\%}$), indicating similar overall performance.

Model	Epoch	Δ Loss	Δ Correlation
CNN	Best (28)	0.1252	0.0506
CNN	Final (50)	0.1677	0.1046
LSTM	Best (48)	0.0716	0.0054
LSTM	Final (50)	0.0875	0.0121

Table 5: Comparison of training and validation metrics for the best CNN and LSTM models. Differences are shown as absolute values: $\Delta = |\text{Train} - \text{Val}|$.

Table 5 reports the absolute differences between training and validation for loss and correlation at both the best and final epochs.

The CNN shows that both correlations keep increasing, while training loss decreases, and validation loss plateaus, indicating a saturation on the model's ability to generalize unseen data. (3)

In contrast, the BiLSTM shows more generalization ability at both best and final epochs, and got better test accuracy at best epoch than the CNN model, enabling the model to achieve higher correlation with less overfitting. The BiLSTM model was then more successful at generalizing the training data to get a better prediction of the binding affinity of an RNA strand with the protein.

Overall, each model has its own advantages for this task. The CNN benefits from its ability to capture local patterns, making it well suited for identifying short RNA motifs directly involved in protein binding. In contrast, the BiLSTM benefits from its ability to model long-range dependencies on the RNA sequence, allowing a better understanding of how the full RNA-protein structural interaction affects binding affinity.

2.2 Attention is all we need

2.2.1 Implementation and expectations.

`homework2_q2.3_kaggle_notebook.ipynb` was used to implement the attention mechanism on LSTM, hoping to squeeze further gains on the Spearman Rank Correlation. While the primary feature an attention head is expected to capture is the core nucleotides binding motif, we also evaluated multi-head attention (MHA) with two heads, treating the number of heads as a tunable hyperparameter.

We expect the attention-enhanced model to get higher test performance at a slight cost of execution time (especially for MHA) (4). The model endowed with the attention mechanism should handle better long range dependencies, by actively taking into account information on all the components of the sequence, instead of hoping that enough compression of the LSTM will allow information not to be lost as we progress along the sequence. Additionally, the mechanism allows the model to assign greater weights to biology relevant features, specially the (U)GCAUG motif, while offering better interpretability through analysis of the learned attention weights. (5)

2.2.2 Results.

Taking the hyperparameters of the best vanilla LSTM, 4 attention-LSTM models were trained using $\boldsymbol{dr} = 0.3$, $\boldsymbol{H} = 256$, and combinations of learning rates, $\boldsymbol{lr} = 3 \times 10^{-4}$ and $\boldsymbol{lr} = 5 \times 10^{-4}$, with 1 and 2 of heads 1 and 2, whose results are contained in `Attention_LSTM_search_results_2.2.json`, which contains the history used to generate table 6 and figure 6.

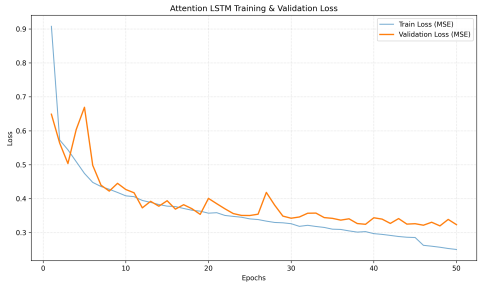
Table 6 shows that the attention models achieved higher validation correlations when compared to CNN and LSTM models. We also notice that the two-head model performed slightly better with $\boldsymbol{lr} = 5 \times 10^{-4}$, corresponding to learning rate of

Heads	LR	Best Val Corr
1	0.0003	0.67778
2	0.0003	0.67118
1	0.0005	0.67811
2	0.0005	0.68042

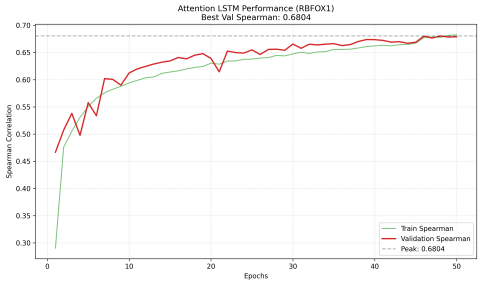
Table 6: Best validation correlation for different numbers of heads and learning rates.

best vanilla LSTM. This is consistent with the idea that in the absence of additional sequence structure beyond the primary binding motif, multiple attention heads are unnecessary and prone to overfitting. To test this hypothesis we will later turn our attention to the heatmaps.

The following plots show the loss and Spearman Rank Correlation for validation and training sets:



(a) Training and Validation Loss



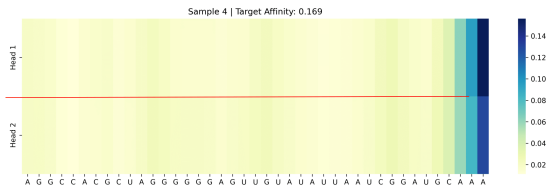
(b) Spearman Rank Correlation

Figure 6: Global performance metrics for the best Attention-LSTM configuration.

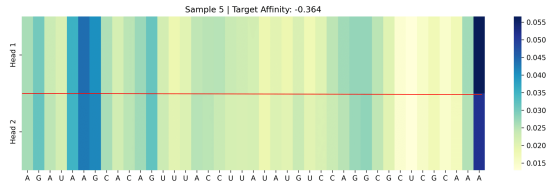
Figure 6 shows us that the training was stable, as the validation spearman curve is monotonic. This also suggests that training time wasn't enough for the model to converge to a maximum correlation, indicating that the attention-LSTM takes more time to converge than the vanilla LSTM. Evaluating the best model for attention-LSTM on the test set, the the model got a Mean Square Error (MSE) loss of 0.3136 and a spearman correlation of 0.6794, within $0,001$ of the best validation value, confirming there is no significant overfitting and the model retained the ability to generalize across unseen data.

Direct comparison of both metrics in figure 6 shows an asymmetry in the time to reach plateau: the loss plateaued before the spearman correlation did, suggesting that the model was quick to predict the general magnitudes of affinities due to the minimization of squared errors, and in later epochs mostly fine-tuned the rankings of these magnitudes, producing small numerical improvements that increase the Spearman correlation. Both correlations keep increasing, while training loss decreases, and validation loss plateaus, indicating a saturation on the model's ability to generalize unseen data. (3)

`homework2_q2.3_kaggle_notebook.ipynb` last two cells generated these 2 sampled (from test set) heatmap visualizations of the attention mechanism for the best performing model with 2 heads:



(a) Sample 4: Focused Attention (+0.169)



(b) Sample 5: Diffuse Attention (-0.364)

Figure 7: Comparison of attention dynamics between high-affinity and low affinity sequences.

The heatmaps shows the second head shows very similar attention patterns, confirming our initial hypothesis that a single head is sufficient to capture the relevant motif. This explains the similarity in performance between the one-head and two-head models in Table 6. Attention visualizations are more informative for complex structures with hierarchically compositional features, but in our case, the high-affinity sequences show focused attention on the nucleotides GCA, while low signal-to-noise sequences exhibit uniform attention.

Overall, the attention-LSTM performs similarly to the vanilla LSTM, suggesting that the RNA sequences are well handled without exceeding LSTM memory limits. Since the motif is localized and distant nucleotides do not interact, multi-head attention provides little additional benefit. An abla-

tion study could further simplify the model without compromising performance.

2.3

2.3.1 Proposed modifications

The protein in question becomes an input to the model, so the data must tell us which protein under what motif sequence got what afinity. For the protein identification we can use one hot encoding if we have zero information on the proteins, treating them as disjoint, which increases by a lot the dimensionality of the input. If, however, we can compress the space of proteins into its components, like amino acids or other relevant characteristics associated with affinity on motifs, the model may grasp a more generalizable ground truth, learning the biochemical latent space. In this case, a combinatorial set of sequence-protein tuple, affinity values for the same sequence under all proteins are contemplated would provide equal grounds so that each sub component of the protein is properly accessed by the model (avoiding data imbalance).

The division of test set should allow some proteins to under any sequence not appear in the training, for example with the same proportion as the sequences are divided in to alow generalization across the sub components of the proteins. The label is still the affinity value but associated with both the specific protein and the sequence. The architecture must allow for two inputs, and we could implement cross attention and feature-wise linear Modulation. (6) The former takes into account the relevance of specific sequences of nucleotides for the protein in question attention, and the latter for the CNN and LSTM takes a linear superposition of both features with parameters derived from the protein on the feature map and hidden states, respectively, thus modulating the model's internal representations.

The evaluation should take into account performance across each protein via an average of the best performance for each protein in low variance of performance per protein scenarios or just getting the average of the average performance per protein across all proteins, in which case the loss should also be weighted so that the model doesn't focus on the more frequent proteins.

2.3.2 Anticipated challenges and benefits

The benefit is again shared representations across proteins, abstracting away from specific proteins and get to the root causal mechanism behind affinity that allows better predictions for proteins with no data. However the model is harder to get right and requires more compute, and we are betting that there is information to be gained from the lower level properties of proteins, or else it would be the same to do one single protein predicting model for every protein.

3 Collaboration with other teams and use of AI tools

AI was used to manage file paths and folder structures on Collab, getting data from team's GitHub repository to collab, and to adapt code for kaggle. All python designed to generate plots from JSON files are sourced by Gemini Pro, and some functions that are commented as ai sourced. ChatGPT offered support on how to correctly use functions from `torch` library, as well as review fundamental concepts such as pooling and batch normalization.

AI assistance was further used for phrasing and wording suggestions during the writing process, having no influence on report's scientific content.

This work was carried out independently, with no collaboration with other teams.

References

[1] Sun S, Zhang Z, Fregoso O, Krainer AR. Mechanisms of activation and repression by the alternative splicing factors RBFOX1/2. *RNA*. 2012;18(2):274–283. doi: 10.1261/rna.030486.111. PubMed PMID: 22184459.

[2] Karpathy, Andrej. "The Unreasonable Effectiveness of Recurrent Neural Networks." Available at: https://karpathy.github.io/2015/05/21/rnn-effectiveness/. Accessed on: January 2, 2026.

[3] Zhang C, Bengio S, Hardt M, Recht B, Vinyals O. Understanding deep learning requires rethinking generalization. arXiv preprint arXiv:1808.01174, 2018. Available at: https://arxiv.org/pdf/1808.01174.

[4] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I. Attention Is All You Need. arXiv preprint arXiv:1706.03762, 2017. Available at: https://arxiv.org/pdf/1706.03762.

[5] S. Xie, D. Chen, R. Zhang, and H. Xue, "Deep Features Analysis with Attention Networks," *AAAI-19 Workshop on Network Interpretability for Deep Learning*, 2019. Available at: https://arxiv.org/abs/1901.10042.

[6] EmergentMind.com. Feature-Wise Linear Modulation (FiLM). Available at: https://www.emergentmind.com/topics/feature-wise-linear-modulation-film. Accessed on: December 2025.